



МИНОБРНАУКИ РОССИИ

**федеральное государственное автономное образовательное учреждение
высшего образования**

«Московский государственный технологический университет

«СТАНКИН»

(ФГАОУ ВО «МГТУ «СТАНКИН»)

Институт
информационных
технологий

Кафедра
информационных систем

**Основная образовательная программа 09.03.02
«Информационные системы и технологии»**

Отчет по дисциплине «Структуры и алгоритмы обработки данных»

Тема: «Лабораторная работа №1»

**Проверил
К.т.н., доцент**

Ковалев И.А.

подпись

**Выполнил
студент группы ИДБ-25-06**

Грачев А.А.

подпись

Москва, 2026 г.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
1. A5. ПЕРЕНОС ПОСЛЕДНЕГО ЭЛЕМЕНТА В НАЧАЛО	4
2. B5. ВТОРОЙ ПО ВЕЛИЧИНЕ ЭЛЕМЕНТ	5
3. C5. ПОМЕНЯТЬ МЕСТАМИ ПОПАРНО	6
4. D5. СЕРЕДИНА СПИСКА ЗА ОДИН ПРОХОД.....	7
5. E5. СЛОЖЕНИЕ ПОЛИНОМОВ	8
ПОЛНЫЙ КОД	10
ЗАКЛЮЧЕНИЕ.....	14

ВВЕДЕНИЕ

Целью работы является изучение операций со связными списками и применение алгоритмов обработки линейных структур данных. В рамках варианта 5 были реализованы пять заданий: перенос последнего элемента в начало, поиск второго максимума, попарный обмен значений, поиск середины списка за один проход и сложение полиномов.

1. A5. ПЕРЕНОС ПОСЛЕДНЕГО ЭЛЕМЕНТА В НАЧАЛО

Последний узел удаляется из конца списка и добавляется в начало. Для односвязного списка необходимо пройти по списку до предпоследнего элемента, поэтому сложность по времени составляет $O(n)$, по памяти - $O(1)$.

Код:

```
def move_last_to_front(ll):  
    last = ll.pop_back()  
    ll.push_front(last)
```

Тесты:

```
# Тест из задания  
ll = create_ll_auto(5)  
ll.print_list()  
# 1 -> 2 -> 3 -> 4 -> 5 -> None  
  
move_last_to_front(ll)  
ll.print_list()  
# 5 -> 1 -> 2 -> 3 -> 4 -> None
```

2. В5. ВТОРОЙ ПО ВЕЛИЧИНЕ ЭЛЕМЕНТ

Во время одного прохода по списку хранятся два значения: максимальное и второе по величине. Дубликаты считаются одним значением, поэтому повторяющиеся максимумы не влияют на результат. Сложность: $O(n)$ по времени и $O(1)$ по памяти.

Код:

```
def second_max(ll):
    first = None
    second = None
    cur = ll.head
    while cur:
        x = cur.data
        if first is None or x > first:
            if x != first:
                second = first
                first = x
            elif x != first and (second is None or x >
second):
                second = x
        cur = cur.next
    return second
```

Тесты:

```
# Тест из задания
ll = create_ll(6)
# Ввод: [5, 3, 9, 9, 1, 7]

ll.print_list()
# 7 -> 1 -> 9 -> 9 -> 3 -> 5 -> None

print(second_max(ll))
# 7

ll = create_ll(3)
# Ввод: [1, 1, 1]

ll.print_list()
# 1 -> 1 -> 1 -> None

print(second_max(ll))
# None
```

3. C5. ПОМЕНЯТЬ МЕСТАМИ ПОПАРНО

Список проходится по два узла за итерацию, после чего значения соседних элементов меняются местами. Так как обход выполняется один раз, сложность составляет $O(n)$ по времени и $O(1)$ по памяти.

Код:

```
def swap_pairs(ll):  
    cur = ll.head  
    while cur and cur.next:  
        cur.data, cur.next.data = cur.next.data, cur.data  
        cur = cur.next.next  
    return ll
```

Тесты:

```
# Тест из задания  
ll = create_ll_auto(5)  
# 1 -> 2 -> 3 -> 4 -> 5 -> None  
swap_pairs(ll)  
ll.print_list()  
# 2 -> 1 -> 4 -> 3 -> 5 -> None
```

4. D5. СЕРЕДИНА СПИСКА ЗА ОДИН ПРОХОД

Для поиска середины используются два указателя: медленный и быстрый. Быстрый продвигается на два узла, медленный - на один. Когда быстрый доходит до конца, медленный указывает на середину. Сложность: $O(n)$ по времени и $O(1)$ по памяти.

Код:

```
def middle_node(ll):
    slow = ll.head
    fast = ll.head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    return slow.data
```

Тесты:

```
# Тест из задания
print(middle_node(build_list([1, 2, 3, 4, 5]))) # 3
print(middle_node(build_list([1, 2, 3, 4])))    # 3
ll = create_ll_auto(5)
ll.print_list()
# 1 -> 2 -> 3 -> 4 -> 5 -> None

print(middle_node(ll))
# 3

ll = create_ll_auto(4)
ll.print_list()
# 1 -> 2 -> 3 -> 4 -> None

print(middle_node(ll))
# 3
```

5. E5. СЛОЖЕНИЕ ПОЛИНОМОВ

Два полинома представлены списками пар (коэффициент, степень) по убыванию степеней. При обходе одновременно сравниваются текущие степени; если степени равны, коэффициенты складываются. Если сумма коэффициентов равна нулю, соответствующий член не добавляется.

Сложность: $O(m + n)$ по времени и $O(m + n)$ по памяти, где m и n - длины входных списков.

Код:

```
class PolyNode:
    def __init__(self, coeff, exp, next=None):
        self.coeff = coeff
        self.exp = exp
        self.next = next

def add_polynomials(p1, p2):
    dummy = PolyNode(0, 0)
    cur = dummy

    while p1 and p2:
        if p1.exp > p2.exp:
            cur.next = PolyNode(p1.coeff, p1.exp)
            p1 = p1.next
        elif p2.exp > p1.exp:
            cur.next = PolyNode(p2.coeff, p2.exp)
            p2 = p2.next
        else:
            new_coeff = p1.coeff + p2.coeff
            if new_coeff != 0:
                cur.next = PolyNode(new_coeff, p1.exp)
            p1 = p1.next
            p2 = p2.next

        if cur.next:
            cur = cur.next

    while p1:
        cur.next = PolyNode(p1.coeff, p1.exp)
        cur = cur.next
        p1 = p1.next

    while p2:
        cur.next = PolyNode(p2.coeff, p2.exp)
        cur = cur.next
```



```
p2 = p2.next
```

```
return dummy.next
```

Тесты:

```
# Тест из задания
p1 = PolyNode(3,4, PolyNode(2,2, PolyNode(1,0)))
p2 = PolyNode(5,3, PolyNode(2,2, PolyNode(4,1)))
result = add_polynomials(p1, p2)
print_poly(result)    # 3x^4 + 5x^3 + 4x^2 + 4x^1 + 1x^0
```

ПОЛНЫЙ КОД

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def push_front(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def push_back(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return

        cur = self.head
        while cur.next:
            cur = cur.next

        cur.next = new_node

    def find(self, data):
        cur = self.head
        while cur:
            if cur.data == data:
                return True
            cur = cur.next
        return False

    def pop_front(self):
        if not self.head:
            return None

        removed_data = self.head.data
        self.head = self.head.next
        return removed_data

    def pop_back(self):
        if not self.head:
            return None

        # если один элемент
        if not self.head.next:
            removed_data = self.head.data
            self.head = None
            return removed_data

        cur = self.head
        while cur.next.next:
            cur = cur.next
        removed_data = cur.next.data
        cur.next = None
        return removed_data

    def print_list(self):
```

```

    cur = self.head
    while cur:
        print(cur.data, end=" -> ")
        cur = cur.next

    print("None")
#E5. Сложение полиномов
class PolyNode:
    def __init__(self, coeff, exp, next=None):
        self.coeff = coeff
        self.exp = exp
        self.next = next
def add_polynomials(p1, p2):
    dummy = PolyNode(0, 0)
    cur = dummy

    while p1 and p2:

        if p1.exp > p2.exp:
            cur.next = PolyNode(p1.coeff, p1.exp)
            p1 = p1.next

        elif p2.exp > p1.exp:
            cur.next = PolyNode(p2.coeff, p2.exp)
            p2 = p2.next

        else:
            # одинаковые степени складываем
            new_coeff = p1.coeff + p2.coeff

            if new_coeff != 0:
                cur.next = PolyNode(new_coeff, p1.exp)

            p1 = p1.next
            p2 = p2.next

        if cur.next:
            cur = cur.next

    # остатки
    while p1:
        cur.next = PolyNode(p1.coeff, p1.exp)
        cur = cur.next
        p1 = p1.next

    while p2:
        cur.next = PolyNode(p2.coeff, p2.exp)
        cur = cur.next
        p2 = p2.next

    return dummy.next
def print_poly(head) -> None:
    cur = head
    parts = []

    while cur:
        parts.append(f"{cur.coeff}x^{cur.exp}")
        cur = cur.next

    print(" + ".join(parts))

```

```

def create_ll(size):
    ll = LinkedList()
    for i in range(size):
        ll.push_front(int(input()))
    return ll
def create_ll_auto(size):
    ll = LinkedList()
    for i in range(size):
        ll.push_front(size - i)
    return ll

#A5. Перенос последнего элемента в начало
def move_last_to_front(ll):
    last = ll.pop_back()
    ll.push_front(last)

#B5. Второй по величине элемент
def second_max(ll):
    first = None
    second = None
    cur = ll.head
    while cur:
        x = cur.data
        if first is None or x > first:
            if x != first:
                second = first
                first = x
            elif x != first and (second is None or x > second):
                second = x
        cur = cur.next
    return second

#C5. Поменять местами попарно
def swap_pairs(ll):
    cur = ll.head
    while cur and cur.next:
        cur.data, cur.next.data = cur.next.data, cur.data
        cur = cur.next.next
    return ll

#D5. Середина списка за один проход
def middle_node(ll):
    slow = ll.head
    fast = ll.head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    return slow.data

ll = create_ll_auto(5)
ll.print_list()

move_last_to_front(ll)
ll.print_list()

ll = create_ll_auto(5)
swap_pairs(ll)
ll.print_list()

ll = create_ll_auto(5)
ll.print_list()

```

```
print(second_max(ll))
print(middle_node(ll))

p1 = PolyNode(3,4, PolyNode(2,2, PolyNode(1,0)))
p2 = PolyNode(5,3, PolyNode(2,2, PolyNode(4,1)))
result = add_polynomials(p1, p2)
print_poly(result)
```

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы были изучены базовые операции над связными списками и способы решения задач за один проход по структуре данных. Выполненные задания показали, как меняется трудоемкость алгоритмов при разных подходах к обработке списка, а также как удобно использовать связный список для хранения полиномов.